

AT32 MCU USB Host Library Application Note

前言

本文档主要描述 AT32F4xx 系列 USB 主机库的架构和使用方法，方便开发者使用库快速开发 USB 相关的应用，同时也会介绍 AT32 BSP 里面对应的 USB HOST 例程。支持库版本 AT32F4xx_Firmware_Library_V2.x.x。

支持型号列表：

支持型号	具备 USB HOST 的型号

目录

1	AT32 USB 主机协议库.....	6
1.1	AT32 USB 库文件	7
2	USB 主机库文件说明	8
2.1	USB 主机文件函数接口	8
2.1.1	usbh_int.c 函数接口.....	8
2.1.2	usbh_core.c 函数接口	8
2.1.3	usbh_ctrl.c 函数接口.....	9
2.1.4	usbh_xx_class.c 函数接口.....	10
2.1.5	usbh_user.c 函数接口.....	11
2.1.6	其它参数	11
2.2	通道 FIFO 分配	13
2.2.1	USB 主机通道 FIFO 分配	13
2.3	USB 主机初始化	13
2.3.1	OTG 主机外设初始化	13
2.4	USB 主机中断处理.....	13
2.4.1	通道中断处理	14
2.4.2	Wakeup 中断处理	15
2.4.3	接收 FIFO 非空中断.....	15
2.4.4	主机端口中断处理.....	16
2.4.5	设备断开连接中断处理	17
2.5	USB 主机枚举处理.....	18
2.5.1	USB 主机枚举流程	19
2.5.2	USB 主机类初始化	23
2.5.3	USB 主机类请求	25
2.5.4	USB 主机数据处理	26
3	USB 主机类型例程.....	28
3.1	HID 例程	28

3.1.1	HID 类例程初始化.....	28
3.1.2	HID 类请求.....	30
3.1.3	HID 类数据处理	31
3.2	MSC 例程	32
3.2.1	MSC 类初始化	33
3.2.2	MSC 类请求.....	34
3.2.3	MSC 类数据处理	35
3.2.4	MSC 读写数据	37
3.3	CDC 例程	38
3.3.1	CDC 类例程初始化.....	38
3.3.2	CDC 类请求.....	41
3.3.3	CDC 类数据处理.....	42
3.4	two_otg_host_demo 例程	43
3.4.1	配置与实现	43
4	版本历史	47

表目录

表 1. USB 主机库文件列表.....	7
表 2. USB 主机类文件列表.....	7
表 3 usbh_int 函数接口.....	8
表 4 usbh_core 函数接口	8
表 5 usbh_ctrl 函数接口.....	9
表 6 标准设备请求	10
表 7 主机 class 函数结构体	10
表 8 主机 class 函数接口	10
表 9 用户事件函数结构体	11
表 10 用户函数接口函数.....	11
表 11 文档版本历史	47

图目录

图 1. USB 主机库结构	6
图 2. AT32 工程结构	7
图 3 USB 主机库文件结构	8
图 4 全局结构体	11
图 5 USB 主机状态机	12
图 6 函数返回值	12
图 7 USB 主机中断处理函数	14
图 8 设备连接状态机流程	19
图 9 USB 主机枚举流程	20
图 10 Get Descriptor	21
图 11 USB 主机 SETUP 函数调用	21
图 12 控制请求处理流程	23

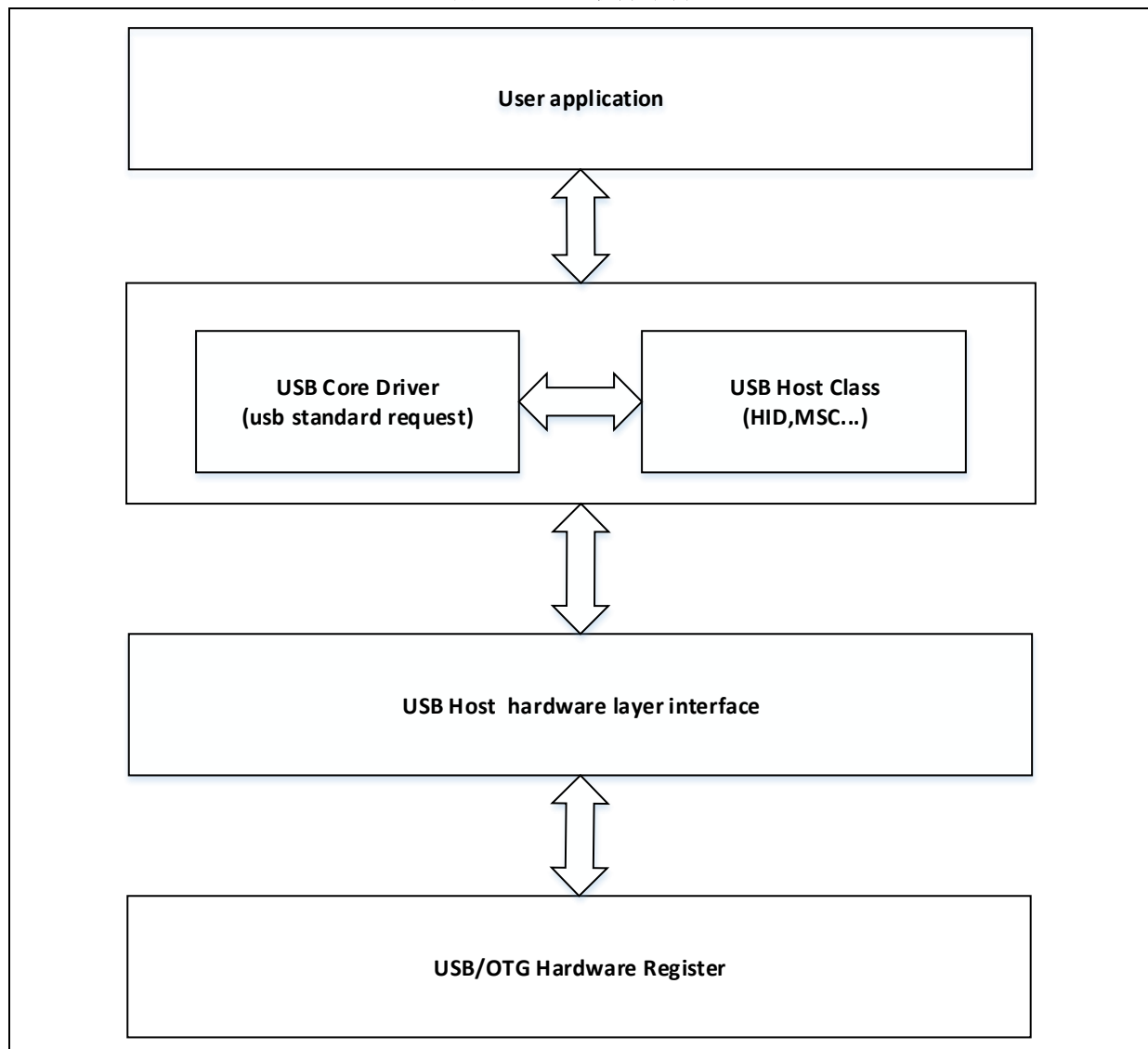
1 AT32 USB 主机协议库

这部分主要介绍AT32 USB主机库的结构和库的使用方法，这里库的作用是用来管理USB外设和实现USB的基本协议，使开发者能够更快的上手开发。

USB 主机库包含以下几个模块 如图1：

- 用户应用程序
此部分为开发者根据应用具体需求开发应用程序。
- USB Core Driver和USB主机类
USB Core Driver：此部分实现USB设备标准协议栈，标准请求等接口。
USB主机类：此部分实现具体USB主机请求。
- USB主机硬件底层接口
此部分实现硬件寄存器抽象接口
- USB/OTG外设

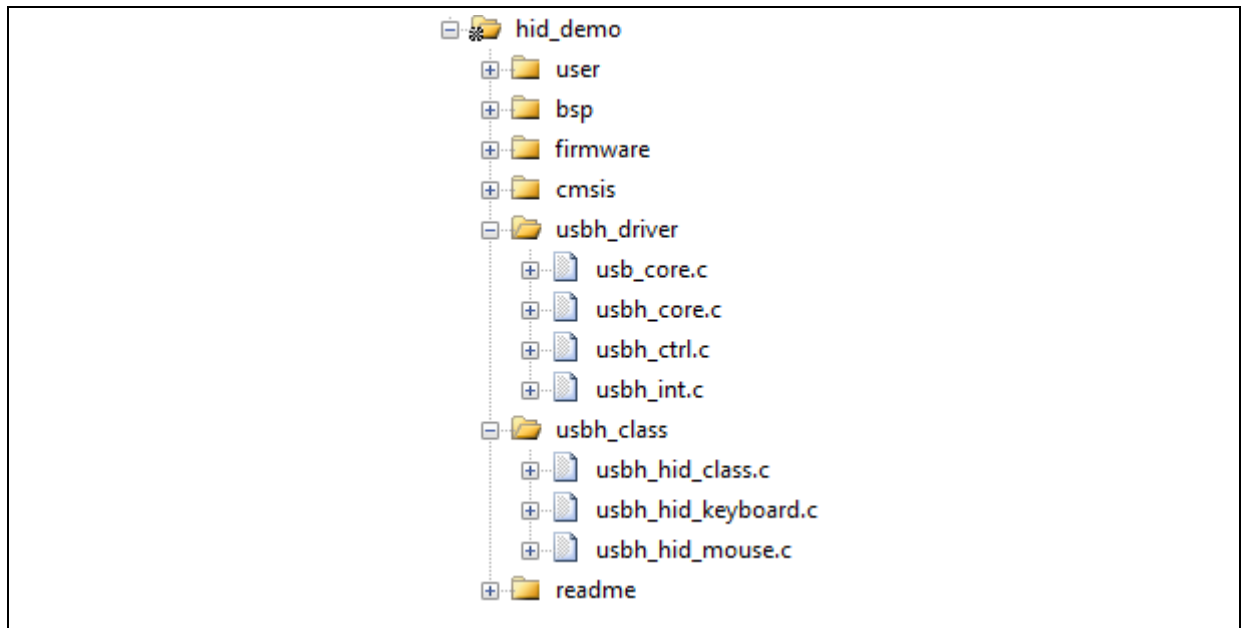
图 1. USB 主机库结构



1.1 AT32 USB 库文件

如下是 AT32 USB 应用工程结构图：

图 2. AT32 工程结构



Core Driver 库路径：OTGFS -->middlewares\usb_drivers

Host Class 库路径：middlewares\usbh_class

如下是 USB 主机库文件列表：

表 1. USB 主机库文件列表

文件名	内容
usb_core.c/h	USB Core
usbh_core.c/h	USB Host Core Library
usbh_int.c/h	USB Host 中断处理
usbh_ctrl.c/h	USB Host 控制请求
usb_std.h	USB 标准头文件
at32f4xx_usb.c/h	USB 硬件寄存器接口

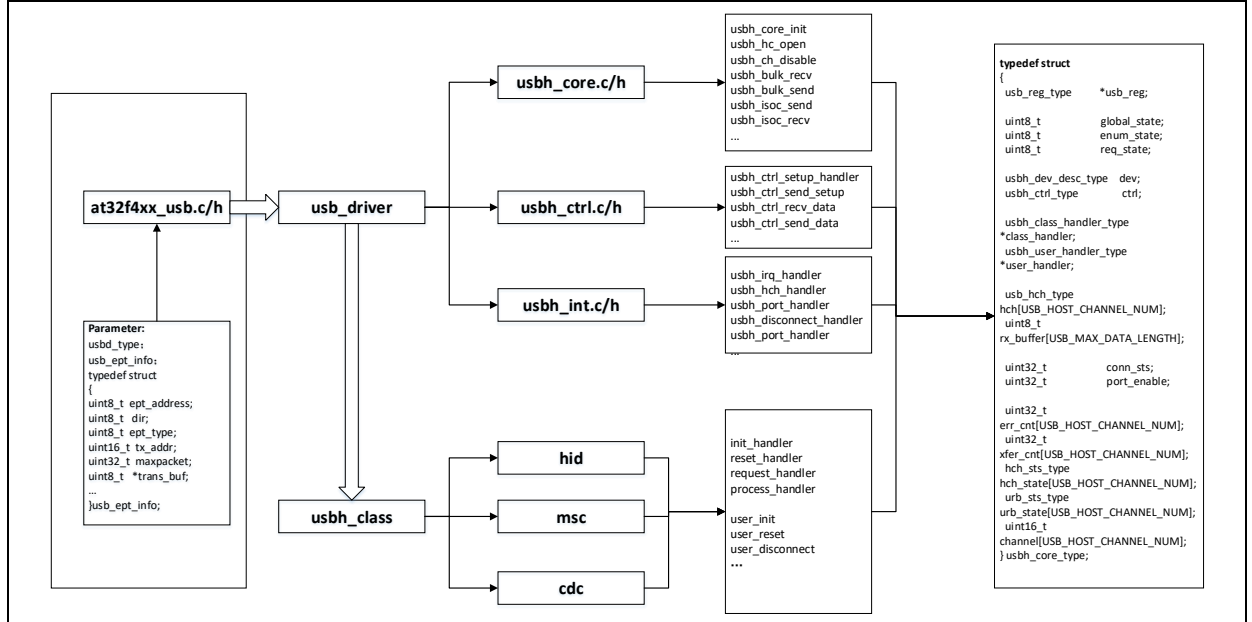
表 2. USB 主机类文件列表

文件名	内容
usbh_hid_class.c/h	Hid 类函数处理
usbh_hid_keyboard.c/h	Hid keyboard 子类处理
usbh_hid_mouse.c/h	Hid mouse 子类处理
usbh_msc_class.c/h	Mass-storage 类函数处理
usbh_msc_bot_scsi.c/h	Bulk-only 传输和 SCSI 命令处理

2 USB 主机库文件说明

USB 库实现 USB 主机枚举标准请求，同时实现 USB 规格里面的 4 种传输类型（control，interrupt，bulk，isochronous）的函数接口。

图 3 USB 主机库文件结构



2.1 USB 主机文件函数接口

2.1.1 usbh_int.c 函数接口

usbh_int.c 主要处理底层中断，包括设备连接，设备断开等。

表 3 usbh_int 函数接口

接口名称	描述
usbd_irq_handler	USB 主机中断入口函数
usbh_hch_handler	USB 主机通道事件中断函数
usbh_port_handler	USB 主机端口事件中断函数
usbh_disconnect_handler	USB 主机连接断开中断函数
usbh_hch_in_handler	USB 主机通道 IN 事件中断函数
usbh_hch_out_handler	USB 主机通道 OUT 事件中断函数
usbh_rx_qlvl_handler	USB 主机接收数据非空中断函数
usbh_wakeup_handler	USB 主机唤醒时间中断
...	...

2.1.2 usbh_core.c 函数接口

usbh_core.c 此文件封装不同的 usb 主机函数接口用于不同的地方调用，包括一些接收，发送函数等。

表 4 usbh_core 函数接口

接口名称	描述
usbh_core_init	USB 主机初始化接口函数

接口名称	描述
usbh_in_out_request	USB 主机 IN/OUT 处理接口函数
usbh_interrupt_recv	USB 主机中断传输接收处理接口函数
usbh_interrupt_send	USB 主机中断传输发送处理接口函数
usbh_bulk_recv	USB 主机 BULK 传输接收处理接口函数
usbh_bulk_send	USB 主机 BULK 传输发送处理接口函数
usbh_isoc_send	USB 主机同步传输发送处理接口函数
usbh_isoc_recv	USB 主机同步传输接收处理接口函数
usbh_cfg_default_init	USB 主机默认状态初始化函数
usbh_enter_suspend	USB 主机进入挂起状态函数
usbh_resume	USB 主机唤醒函数
usbh_alloc_channel	USB 主机通道分配函数
usbh_get_urb_status	USB 主机获取通道传输状态函数
usbh_ctrl_result_check	USB 主机获取 SETUP 传输状态函数
usbh_alloc_address	USB 主机分配设备地址函数
usbh_reset_port	USB 主机复位端口函数
usbh_loop_handler	USB 主机主循环函数
usbh_ch_disable	USB 主机通道关闭
usbh_hc_open	USB 主机通道打开
...	...

2.1.3 usbh_ctrl.c 函数接口

usbh_ctrl.c 此文件处理 USB 主机枚举过程中的一些标准请求。

表 5 usbh_ctrl 函数接口

接口名称	描述
usbh_ctrl_send_setup	SETUP 请求发送函数
usbh_ctrl_recv_data	控制传输接收数据函数
usbh_ctrl_send_data	控制传输发送数据函数
usbh_ctrl_setup_handler	SETUP 传输处理
usbh_ctrl_transfer_loop	控制传输循环
usbh_ctrl_request	使能一个控制传输请求
usbh_get_descriptor	获取设备描述
usbh_parse_dev_desc	解析设备描述函数
usbh_parse_interface_desc	解析设备接口描述
usbh_parse_endpoint_desc	解析设备端点描述
usbh_parse_configure_desc	解析设备配置描述
usbh_find_interface	查找对应接口
usbh_parse_string_desc	解析设备字符串描述
usbh_get_device_descriptor	获取设备描述信息
usbh_get_configure_descriptor	获取设备配置信息
usbh_get_string_descriptor	获取设备厂商，产品等描述信息
usbh_set_configuration	SET CONFIGURATION 接口
usbh_set_address	SET ADDRESS 接口

接口名称	描述
usbh_set_interface	SET INTERFACE 接口
usbh_set_feature	SET FEATURE 接口
usbh_clear_dev_feature	CLEAR FEATURE 接口 (device)
usbh_clear_ept_feature	CLEAR FEATURE 接口 (endpoint)
...	...

支持的标准设备请求如下表：

表 6 标准设备请求

请求	描述
GET_STATUS	获取状态
CLEAR_FEATURE	清除 feature
SET_FEATURE	设置 feature
SET_ADDRESS	设置设备地址
GET_DESCRIPTOR	获取设备描述
GET_CONFIGURATION	获取设备配置
SET_CONFIGURATION	设置设备配置
GET_INTERFACE	获取 alternate setting
SET_INTERFACE	设置 alternate setting

2.1.4 usbh_xx_class.c 函数接口

usbh_xx_class.c 此文件为具体主机类型的数据处理，通过结构体函数来实现不同主机类型数据的处理。开发者根据不同的主机类型，来实现下面函数结构体中的函数，达到不同应用结果。

函数结构体如下：

表 7 主机 class 函数结构体

<pre>/** * @brief host class handler type */ typedef struct { usb_sts_type (*init_handler)(void *uhost); /*!< usb host class init handler */ usb_sts_type (*reset_handler)(void *uhost); /*!< usb host class reset handler */ usb_sts_type (*request_handler)(void *uhost); /*!< usb host class request handler */ usb_sts_type (*process_handler)(void *uhost); /*!< usb host class process handler */ void *pdata; /*!< usb host class data */ } usbh_class_handler_type;</pre>	
--	--

表 8 主机 class 函数接口

接口函数	描述
init_handler	USB 主机类初始化接口
reset_handler	USB 主机类复位接口
request_handler	USB 主机类控制请求接口
process_handler	USB 主机数据处理函数接口

2.1.5 usbh_user.c 函数接口

usbh_user.c 此文件主要为一些给用户处理事件的函数如复位，断开连接等。

表 9 用户事件函数结构体

typedef struct		
{		
usb_sts_type (*user_init)(void);		/*!< usb host user init handler */
usb_sts_type (*user_reset)(void);		/*!< usb host user reset handler */
usb_sts_type (*user_attached)(void);		/*!< usb host user attached handler */
usb_sts_type (*user_disconnect)(void);		/*!< usb host user disconnect handler */
usb_sts_type (*user_speed)(uint8_t speed);		/*!< usb host user speed handler */
usb_sts_type (*user_mfc_string)(void *);		/*!< usb host user manufacturer string handler */
usb_sts_type (*user_product_string)(void *);		/*!< usb host user product string handler */
usb_sts_type (*user_serial_string)(void *);		/*!< usb host user serial handler */
usb_sts_type (*user_enumeration_done)(void);		/*!< usb host user enumeration done handler */
usb_sts_type (*user_application)(void);		/*!< usb host user application handler */
usb_sts_type (*user_active_vbus)(void *uhost, confirm_state state);		/*!< usb host user active vbus */
usb_sts_type (*user_not_support)(void);		/*!< usb host user not support handler */
} usbh_user_handler_type;		

表 10 用户函数接口函数

接口函数	描述
usbh_user_init	用户代码初始化
usbh_user_reset	用户代码复位
usbh_user_attached	设备 attached 事件
usbh_user_disconnect	设备断开连接
usbh_user_speed	连接设备枚举速度(全速，低速)
usbh_user_mfc_string	设备厂商信息
usbh_user_product_string	设备产品信息信息
usbh_user_serial_string	设备序列号信息
usbh_user_enumeration_done	枚举完成
usbh_user_application	用户应用处理
usbh_user_active_vbus	VBUS 供电使能
usbh_user_not_support	不支持的处理

2.1.6 其它参数

函数的参数结构体如下，USB 主机库中参数传递使用结构体 usbh_core_type，如下图：

图 4 全局结构体

typedef struct		
{		
usb_reg_type	*usb_reg;	/*!< usb register pointer */
uint8_t	global_state;	/*!< usb host global state machine */
uint8_t	enum_state;	/*!< usb host enumeration state machine */

```

uint8_t                req_state;                /*!< usb host request state machine */

usbh_dev_desc_type      dev;                     /*!< usb device descriptor */
usbh_ctrl_type          ctrl;                    /*!< usb host control transfer struct */
usbh_class_handler_type *class_handler;          /*!< usb host class handler pointer */
usbh_user_handler_type  *user_handler;           /*!< usb host user handler pointer */
usb_hch_type            hch[USB_HOST_CHANNEL_NUM]; /*!< usb host channel array */
uint8_t                 rx_buffer[USB_MAX_DATA_LENGTH]; /*!< usb host rx buffer */
uint32_t                conn_sts;                /*!< connect status */
uint32_t                port_enable;              /*!< port enable status */
uint32_t                err_cnt[USB_HOST_CHANNEL_NUM]; /*!< error counter */
uint32_t                xfer_cnt[USB_HOST_CHANNEL_NUM]; /*!< xfer counter */
hch_sts_type            hch_state[USB_HOST_CHANNEL_NUM]; /*!< channel state */
urb_sts_type            urb_state[USB_HOST_CHANNEL_NUM]; /*!< usb request state */
uint16_t                channel[USB_HOST_CHANNEL_NUM]; /*!< channel array */
} usbh_core_type;

```

USB 主机的状态机如下图：

图 5 USB 主机状态机

```

typedef enum
{
    USBH_IDLE,                /*!< usb host global state idle */
    USBH_PORT_EN,             /*!< usb host global state port enable */
    USBH_ATTACHED,            /*!< usb host global state attached */
    USBH_DISCONNECT,          /*!< usb host global state disconnect */
    USBH_DEV_SPEED,           /*!< usb host global state device speed */
    USBH_ENUMERATION,         /*!< usb host global state enumeration */
    USBH_CLASS_REQUEST,       /*!< usb host global state class request */
    USBH_CLASS,               /*!< usb host global state class */
    USBH_CTRL_XFER,           /*!< usb host global state control transfer */
    USBH_USER_HANDLER,        /*!< usb host global state user handler */
    USBH_SUSPEND,             /*!< usb host global state suspend */
    USBH_SUSPENDED,           /*!< usb host have in suspend mode */
    USBH_WAKEUP,              /*!< usb host global state wakeup */
    USBH_UNSUPPORT,           /*!< usb host global unsupport device */
    USBH_ERROR_STATE,         /*!< usb host global state error */
} usbh_gstate_type;

```

USB 返回值，USB 函数接口使用如下函数返回值。

图 6 函数返回值

```

typedef enum
{
    USB_OK,                   /*!< usb status ok */
    USB_FAIL,                 /*!< usb status fail */
    USB_WAIT,                 /*!< usb status wait */
}

```

```
USB_NOT_SUPPORT, /*!< usb status not support */
USB_ERROR,        /*!< usb status error */
}usb_sts_type;
```

2.2 通道 FIFO 分配

USB 主机要正常收发数据，在初始化时需要为通道分配 FIFO，分配给所有通道 FIFO 大小的和不能超过系统分配给 USB 缓冲区的最大长度，具体 USB 的缓冲区大小参考 RM 上的描述。开发者可以参考例程 `usb_conf.h` 为通道自定义分配 FIFO。

2.2.1 USB 主机通道 FIFO 分配

主机模式下，所有通道共享一个接收 FIFO，通道发送 FIFO 分为非周期性发送 FIFO 和周期性发送 FIFO。注意 `usb_conf.h` 中对端点分配的 FIFO 大小单位为 word（Byte）。

以 `hid_demo` 例程为例：

```
#define USBH_RX_FIFO_SIZE          128 /*共享的一个接收 FIFO，大小 128*4 Byte */
#define USBH_NP_TX_FIFO_SIZE       96 /*非周期性发送 FIFO，大小 96*4 Byte */
#define USBH_P_TX_FIFO_SIZE        96 /*周期性发送 FIFO，大小 96*4 Byte */
```

2.3 USB 主机初始化

在使用 USB 主机功能时，需要对 USB 的寄存器做一些基本的初始化，通过调用 USB 主机的初始化函数完成这部分的操作。

2.3.1 OTG 主机外设初始化

OTG 主机初始化函数 `usbh_init` 包含 5 个参数：

```
usb_sts_type usbh_init(otg_core_type *otgdev,
                       uint8_t core_id, uint8_t usb_id,
                       usbh_class_handler_type *class_handler,
                       usbh_user_handler_type *user_handler)
```

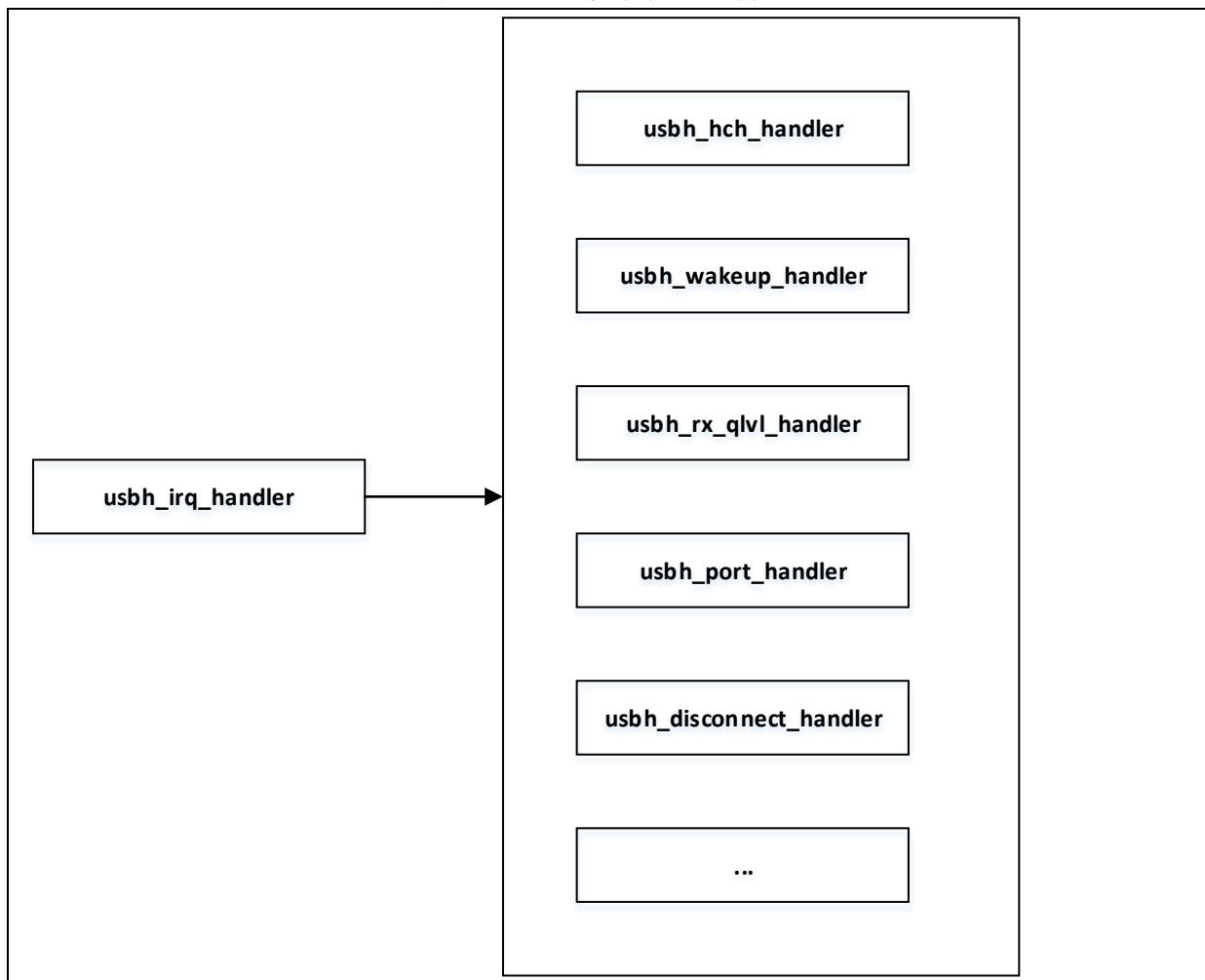
例程主机 `hid_demo` 的初始化如下：

```
usbh_init(&otg_core_struct,
          USB_FULL_SPEED_CORE_ID,
          USB_ID,
          &uhost_hid_class_handler,
          &usbh_user_handle);
```

2.4 USB 主机中断处理

USB 主机中断入口函数 `usbh_irq_handler` 处理所有 USB 主机中断，包括连接状态，通道收发数据，SOF，唤醒等中断，下面介绍一些典型的中断处理。

图 7 USB 主机中断处理函数



2.4.1 通道中断处理

当 USB 主机与设备进行通信时，通道的收发状态都在通道中断中处理，通道的方向分为 IN 和 OUT 通过通道寄存器中的方向为进行判断。

中断处理函数：usbh_hch_handler

代码如下：

```
void usbh_hch_handler(usbh_core_type *uhost)
{
    otg_global_type *usbx = uhost->usb_reg;
    otg_host_type *usb_host = OTG_HOST(usbx);
    uint32_t intsts, i_index;

    intsts = usb_host->haint & 0xFFFF;
    for(i_index = 0; i_index < 16; i_index++)
    {
        if(intsts & (1 << i_index))
        {
            if(USB_CHL(usbx, i_index)->hcchar_bit.eptdir)
            {
```

```
        //hc in
        usbh_hch_in_handler(uhost, i_index);
    }
    else
    {
        //hc out
        usbh_hch_out_handler(uhost, i_index);
    }
}
}
```

2.4.2 Wakeup 中断处理

当 USB 主机在挂起状态时，检测到设备触发的远程唤醒信号，将产生此中断。此中断会将当前主机状态改为 WAKEUP 状态：

中断处理函数：usbh_wakeup_handler

代码如下：

```
void usbh_wakeup_handler(usbh_core_type *uhost)
{
    uhost->global_state = USBH_WAKEUP;
}
```

2.4.3 接收 FIFO 非空中断

此中断表示当前接收 FIFO 有数据，应用程序通过此中断读出接收到的数据，并存放到对应通道的 FIFO 当中，当数据读取完成之后，会产生对应的通道的中断。

中断处理函数：usbh_rx_qlvl_handler

代码如下：

```
void usbh_rx_qlvl_handler(usbh_core_type *uhost)
{
    uint8_t chn;
    uint32_t pktsts;
    uint32_t pktcnt;
    uint32_t tmp;
    otg_hchannel_type *ch;
    otg_global_type *usbx = uhost->usb_reg;

    usbx->gintmsk_bit.rxfvlmsk = 0;

    tmp = usbx->grxstsp;
    chn = tmp & 0xF;
    pktsts = (tmp >> 17) & 0xF;
    pktcnt = (tmp >> 4) & 0x7FFF;
    ch = USB_CHL(usbx, chn);
```

```
switch(pktsts)
{
    case PKTSTS_IN_DATA_PACKET_RECV:
        if(pktcnt > 0 && (uhost->hch[chn].trans_buf != 0))
        {
            usb_read_packet(usbx, uhost->hch[chn].trans_buf, chn, pktcnt);
            uhost->hch[chn].trans_buf += pktcnt;
            uhost->hch[chn].trans_count += pktcnt;

            if(ch->hctsiz_bit.pktcnt > 0)
            {
                ch->hcchar_bit.chdis = FALSE;
                ch->hcchar_bit.chena = TRUE;
                uhost->hch[chn].toggle_in ^= 1;
            }
        }
        break;
    case PKTSTS_IN_TRANSFER_COMPLETE:
        break;
    case PKTSTS_DATA_BIT_ERROR:
        break;
    case PKTSTS_CHANNEL_STOP:
        break;
    default:
        break;
}
usbx->gintmsk_bit.rxfvlmsk = 1;
}
```

2.4.4 主机端口中断处理

当 USB 主机断开发生状态改变时，会产生端口中断，应用程序通过此中断判断连接设备的速度等。

中断处理函数：usbh_port_handler

```
void usbh_port_handler(usbh_core_type *uhost)
{
    otg_global_type *usbx = uhost->usb_reg;
    otg_host_type *usb_host = OTG_HOST(usbx);

    uint32_t prt = 0, prt_0;

    prt = usb_host->hpert;
    prt_0 = prt;

    prt_0 &= ~(USB_OTG_HPRT_PRTENA | USB_OTG_HPRT_PRTENCHNG |
              USB_OTG_HPRT_PRTOVRCAC | USB_OTG_HPRT_PRTCONDET);
    if(prt & USB_OTG_HPRT_PRTCONDET)
```



```

{
    if(prt & USB_OTG_HPRT_PRTCONSTS)
    {
        /* connect callback */
        uhost->conn_sts = 1;
    }
    prt_0 |= USB_OTG_HPRT_PRTCONDET;
}

if(prt & USB_OTG_HPRT_PRTENCHNG)
{
    prt_0 |= USB_OTG_HPRT_PRTENCHNG;

    if(prt & USB_OTG_HPRT_PRTENA)
    {
        if((prt & USB_OTG_HPRT_PRTSPD) == (USB_PRTSPD_LOW_SPEED << 17))
        {
            usbh_fsfs_clkssel(usbh, USB_HCFG_CLK_6M);
        }
        else
        {
            usbh_fsfs_clkssel(usbh, USB_HCFG_CLK_48M);
        }
        /* connect callback */
        uhost->port_enable = 1;
    }
    else
    {
        /* clean up hprt */
        usb_host->hprt &= ~(USB_OTG_HPRT_PRTENA | USB_OTG_HPRT_PRTENCHNG |
            USB_OTG_HPRT_PRTOVRCAC | USB_OTG_HPRT_PRTCONDET);
    }
}

if(prt & USB_OTG_HPRT_PRTOVRCAC)
{
    prt_0 |= USB_OTG_HPRT_PRTOVRCAC;
}

usb_host->hprt = prt_0;
}

```

2.4.5 设备断开连接中断处理

当连接的设备断开是，会产生断开连接中断。

中断处理函数：usbh_disconnect_handler

代码:

```
void usbh_disconnect_handler(usbh_core_type *uhost)
{
    otg_global_type *usbh = uhost->usb_reg;
    otg_host_type *usb_host = OTG_HOST(usbh);
    uint8_t i_index;

    usb_host_disable(usbh);

    uhost->conn_sts = 0;

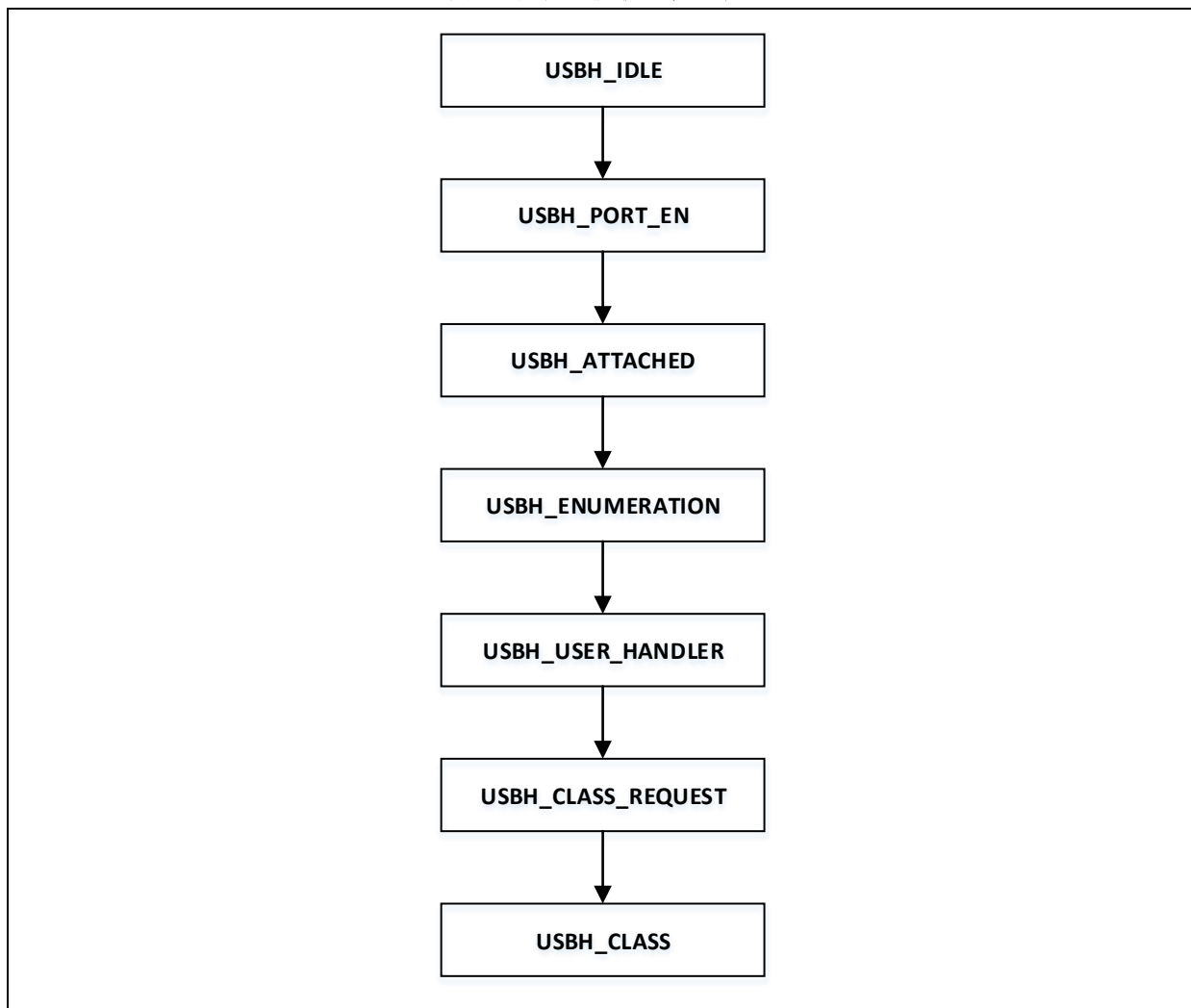
    uhost->global_state = USBH_DISCONNECT;

    for(i_index = 0; i_index < USB_HOST_CHANNEL_NUM; i_index++)
    {
        usbh_free_channel(uhost, i_index);
    }
    usbh_fsls_clkssel(usbh, USB_HCFG_CLK_48M);
}
```

2.5 USB 主机枚举处理

USB 主机状态机在 `usbh_loop_handler` 函数中处理，对应的主机状态机 `uhost->global_state`，如下图所示是设备连接到主机的状态处理流程：

图 8 设备连接状态机流程



- USBH_IDLE: 当 USB 启动或者设备断开连接之后在此状态
- USBH_PORT_EN: USB 主机端口使能之后在此状态
- USBH_ATTACHED: USB 设备连接稳定之后
- USBH_ENUMERATION: USB 设备标准的枚举流程
- USBH_USER_HANDLER: 枚举完成之后，调用主机类的初始化函数
- USBH_CLASS_REQUEST: 主机类初始化完成之后，进行主机类请求
- USBH_CLASS: 所有枚举完成，进行正常的数据处理。

2.5.1 USB 主机枚举流程

当设备接到主机之后，通过控制端点（端点 0）进行枚举动作，USB 主机的枚举状态如下：

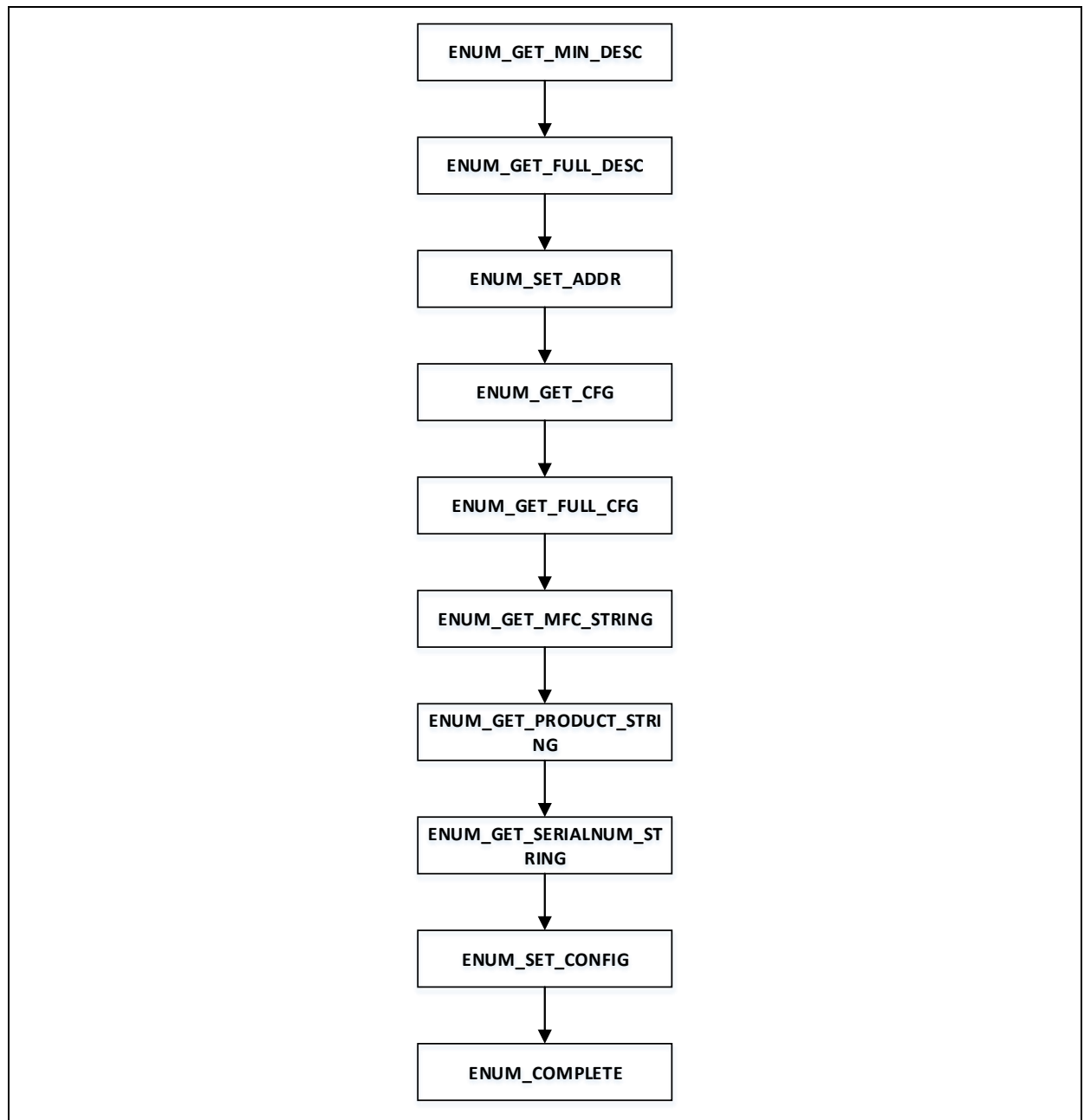
```
typedef enum
{
    ENUM_IDLE,                                /*!< usb host enumeration state idle */
    ENUM_GET_MIN_DESC,                        /*!< usb host enumeration state get descriptor 8 byte*/
    ENUM_GET_FULL_DESC,                      /*!< usb host enumeration state get descriptor 18 byte*/
    ENUM_SET_ADDR,                           /*!< usb host enumeration state set address */
    ENUM_GET_CFG,                            /*!< usb host enumeration state get configuration */
}
```

ENUM_GET_FULL_CFG,	/*!< usb host enumeration state get full configuration */
ENUM_GET_MFC_STRING,	/*!< usb host enumeration state get manufacturer string */
ENUM_GET_PRODUCT_STRING,	/*!< usb host enumeration state get product string */
ENUM_GET_SERIALNUM_STRING,	/*!< usb host enumeration state get serial number string */
ENUM_SET_CONFIG,	/*!< usb host enumeration state set config */
ENUM_COMPLETE,	/*!< usb host enumeration state complete */

} usbh_enum_sts_type;

如下是主机枚举的状态机流程:

图 9 USB 主机枚举流程

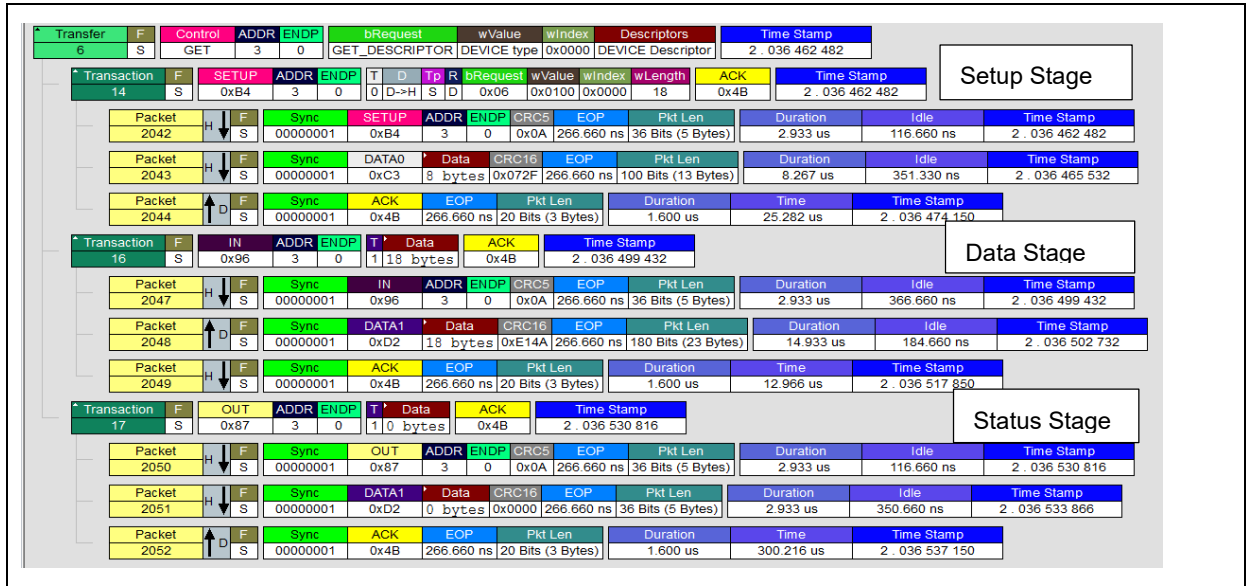


- **ENUM_GET_MIN_DESC**: 获取 8 字节设备描述
- **ENUM_GET_FULL_DESC**: 获取 18 字节设备描述
- **ENUM_SET_ADDR**: 设置设备地址
- **ENUM_GET_CFG**: 获取 9 字节配置描述
- **ENUM_GET_FULL_CFG**: 获取完整的配置描述

- ENUM_GET_MFC_STRING: 获取设备厂商信息
- ENUM_GET_PRODUCT_STRING: 获取设备产品信息
- ENUM_GET_SERIALNUM_STRING: 获取设备序列号信息
- ENUM_SET_CONFIG: SET CONFIGURE
- ENUM_COMPLETE: 枚举完成

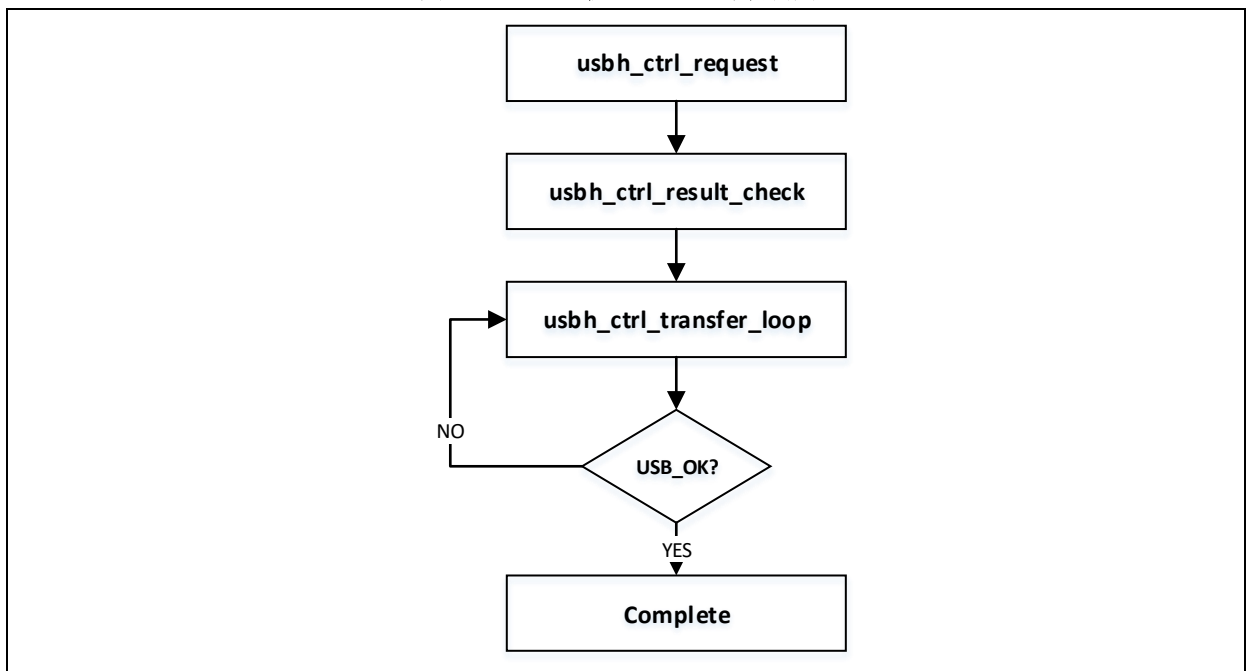
USB 控制传输过程包含 SETUP-DATA-STATUS 三个阶段，如下是一个主机获取设备信息的传输过程 GET_DESCRIPTOR:

图 10 Get Descriptor



如下是 USB 主机库一个 SETUP-DATA-STATUS 函数调用流程:

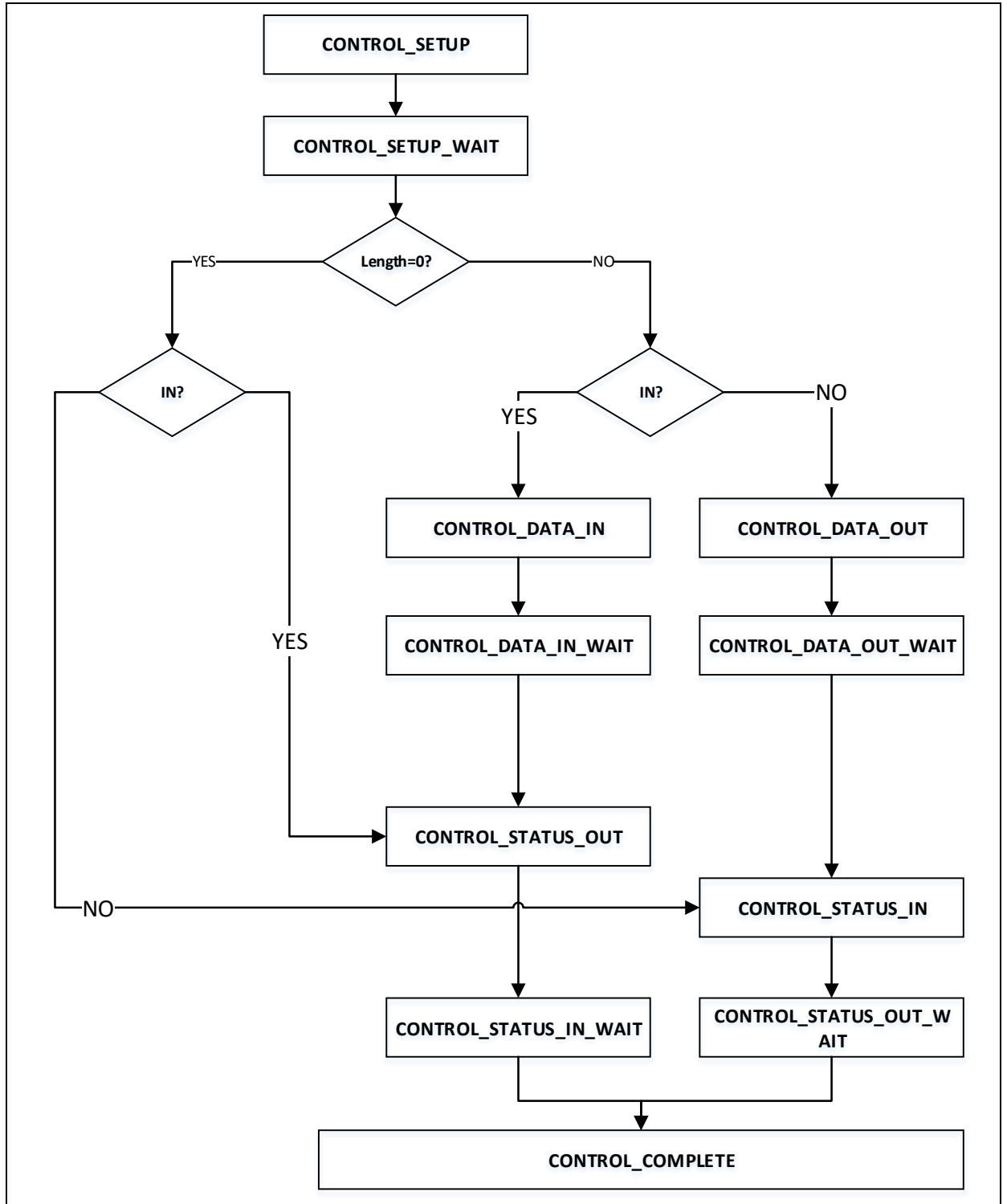
图 11 USB 主机 SETUP 函数调用



usbh_ctrl_transfer_loop 函数中断对控制请求的各个状态阶段进行处理:
控制请求状态机:

```
typedef enum
{
    CONTROL_IDLE,           /*!< usb control state idle */
    CONTROL_SETUP,          /*!< usb control state setup */
    CONTROL_SETUP_WAIT,     /*!< usb control state setup wait */
    CONTROL_DATA_IN,        /*!< usb control state data in */
    CONTROL_DATA_IN_WAIT,   /*!< usb control state data in wait */
    CONTROL_DATA_OUT,       /*!< usb control state data out */
    CONTROL_DATA_OUT_WAIT,  /*!< usb control state data out wait */
    CONTROL_STATUS_IN,      /*!< usb control state status in */
    CONTROL_STATUS_IN_WAIT, /*!< usb control state status in wait */
    CONTROL_STATUS_OUT,     /*!< usb control state out */
    CONTROL_STATUS_OUT_WAIT, /*!< usb control state out wait */
    CONTROL_ERROR,          /*!< usb control state error */
    CONTROL_STALL,          /*!< usb control state stall */
    CONTROL_COMPLETE        /*!< usb control state complete */
} ctrl_ept0_sts_type;
```

图 12 控制请求处理流程



2.5.2 USB 主机类初始化

在 USB 标准枚举完成之后，会调用 USB 主机类的初始化，在初始化函数中解析当前枚举的设备是否支持，调用函数 `uhost->class_handler->init_handler(uhost)`。

初始化代码例：

```

usb_sts_type uhost_init_handler(void *uhost)
{

```

```

usbh_core_type *puhost = (usbh_core_type *)uhost;
usb_sts_type status = USB_OK;
uint8_t hidx, eptidx = 0;
usbh_hid_type *phid = &usbh_hid;

puhost->class_handler->pdata = &usbh_hid;

/* get hid interface */
hidx = usbh_find_interface(puhost, USB_CLASS_CODE_HID, 0x01, 0xFF);
if(hidx == 0xFF)
{
    USBH_DEBUG("Unsupport Device!");
    return USB_NOT_SUPPORT;
}

/* get hid protocol */
phid->protocol = puhost->dev.cfg_desc.interface[hidx].interface.bInterfaceProtocol;

if(phid->protocol == USB_HID_MOUSE_PROTOCOL_CODE)
{
    USBH_DEBUG("Mouse Device!");
}
else if(phid->protocol == USB_HID_KEYBOARD_PROTOCOL_CODE)
{
    USBH_DEBUG("Keyboard Device!");
}

for(eptidx = 0; eptidx < puhost->dev.cfg_desc.interface[hidx].interface.bNumEndpoints; eptidx++)
{
    if(puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress & 0x80)
    {
        /* find interface out endpoint information */
        phid->eptin = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress;
        phid->in_maxpacket = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].wMaxPacketSize;
        phid->in_poll = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bInterval;

        phid->chin = usbh_alloc_channel(puhost, phid->eptin);
        /* enable channel */
        usbh_hc_open(puhost, phid->chin, phid->eptin,
                    puhost->dev.address, EPT_INT_TYPE,
                    phid->in_maxpacket,
                    puhost->dev.speed);
    }
    else
    {
        /* get interface out endpoint information */
    }
}

```



```
phid->eptout = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress;
phid->out_maxpacket = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].wMaxPacketSize;
phid->out_poll = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bInterval;

phid->chout = usbh_alloc_channel(puhost, usbh_hid.eptout);
/* enable channel */
usbh_hc_open(puhost, phid->chout, phid->eptout,
             puhost->dev.address, EPT_INT_TYPE,
             phid->out_maxpacket,
             puhost->dev.speed);
}
}
phid->ctrl_state = USB_HID_STATE_IDLE;
return status;
}
```

2.5.3 USB 主机类请求

USB 主机类初始化之后，可以发送主机类控制请求，根据不同的设备类，需要实现不同的设备类请求。调用函数 `uhost->class_handler->request_handler((void *)uhost)`;

主机类请求例：

```
usb_sts_type uhost_request_handler(void *uhost)
{
    usb_sts_type status = USB_WAIT;
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_hid_type *phid = (usbh_hid_type *)puhost->class_handler->pdata;

    switch(phid->ctrl_state)
    {
        case USB_HID_STATE_IDLE:
            phid->ctrl_state = USB_HID_STATE_GET_DESC;
            break;
        case USB_HID_STATE_GET_DESC:
            if(usbh_hid_get_desc(puhost, 9) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_GET_REPORT;
            }
            break;
        case USB_HID_STATE_GET_REPORT:
            if(usbh_hid_get_report(puhost, phid->hid_desc.wItemLength) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_SET_IDLE;
            }
            break;
        case USB_HID_STATE_SET_IDLE:
```

```

        if(usbh_hid_set_idle(puhost, 0, 0) == USB_OK)
        {
            phid->ctrl_state = USB_HID_STATE_SET_PROTOCOL;
        }
        break;
case USB_HID_STATE_SET_PROTOCOL:
    if(usbh_hid_set_protocol(puhost, 0) == USB_OK)
    {
        phid->ctrl_state = USB_HID_STATE_COMPLETE;
    }
    break;
case USB_HID_STATE_COMPLETE:
    phid->state = USB_HID_INIT;
    status = USB_OK;
    break;
default:
    break;
}

return status;
}

```

2.5.4 USB 主机数据处理

所有枚举完成之后，将进行主机数据处理

主机数据处理例：

```

usb_sts_type uhost_process_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_hid_type *phid = (usbh_hid_type *)puhost->class_handler->pdata;
    urb_sts_type urb_status;
    switch(phid->state)
    {
        case USB_HID_INIT:
            phid->state = USB_HID_GET;
            break;

        case USB_HID_GET:
            usbh_interrupt_rcv(puhost, phid->chin, phid->buffer, phid->in_maxpacket);
            phid->state = USB_HID_POLL;
            phid->poll_timer = usbh_get_frame(puhost->usb_reg);
            break;

        case USB_HID_POLL:
            if((usbh_get_frame(puhost->usb_reg) - phid->poll_timer) >= phid->in_poll )
            {

```

```

        phid->state = USB_HID_GET;
    }
    else
    {
        urb_status = usbh_get_urb_status(puhost, phid->chin);
        if(urb_status == URB_DONE)
        {
            puhost->urb_state[phid->chin] = URB_IDLE;
            if(phid->protocol == USB_HID_MOUSE_PROTOCOL_CODE)
            {
                usbh_hid_mouse_decode(phid->buffer);
            }
            else if(phid->protocol == USB_HID_KEYBOARD_PROTOCOL_CODE)
            {
                usbh_hid_keyboard_decode(phid->buffer);
            }
        }
        else if(urb_status == URB_STALL)
        {
            if(usbh_clear_endpoint_feature(puhost, phid->eptin, phid->chin) == USB_OK)
            {
                phid->state = USB_HID_GET;
            }
        }
    }
    break;

default:
    break;
}
return USB_OK;
}

```

3 USB 主机类型例程

本章将说明使用 AT32 USB 主机库实现不同的主机类型的例程。目前实现的主机类例程如下：

- hid (mouse,keyboard)
- mass storage (msc_only_fat32)
- two_otg_host_demo (仅 AT32F435/437 支持)

3.1 HID 例程

HID 类支持鼠标和键盘设备，开发者可修改此例程用以实现支持其它 HID 类设备。

HID 类：

```
usbh_class_handler_type uhost_class_handler =
{
    uhost_init_handler,
    uhost_reset_handler,
    uhost_request_handler,
    uhost_process_handler,
    &usbh_hid
};
```

3.1.1 HID 类例程初始化

当 USB 设备标准枚举完成之后，会进行初始化，调用 `uhost_init_handler`，调用此函数将解析接到主机上的设备类型，是鼠标还是键盘或者其它设备。

通过设备的配置描述信息进行如下流程解析：

- 查找 HID 接口
- 判断接口是鼠标还是键盘
- 查找对应接口的端点号
- 初始化对应通道

代码如下：

```
usb_sts_type uhost_init_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usb_sts_type status = USB_OK;
    uint8_t hidx, eptidx = 0;
    usbh_hid_type *phid = &usbh_hid;

    puhost->class_handler->pdata = &usbh_hid;

    /* get hid interface */
    hidx = usbh_find_interface(puhost, USB_CLASS_CODE_HID, 0x01, 0xFF);
    if(hidx == 0xFF)
    {
        USBH_DEBUG("Unsupport Device!");
    }
}
```

```

return USB_NOT_SUPPORT;
}

/* get hid protocol */
phid->protocol = puhost->dev.cfg_desc.interface[hidx].interface.bInterfaceProtocol;

if(phid->protocol == USB_HID_MOUSE_PROTOCOL_CODE)
{
    USBH_DEBUG("Mouse Device!");
}
else if(phid->protocol == USB_HID_KEYBOARD_PROTOCOL_CODE)
{
    USBH_DEBUG("Keyboard Device!");
}

for(eptidx = 0; eptidx < puhost->dev.cfg_desc.interface[hidx].interface.bNumEndpoints; eptidx++)
{
    if(puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress & 0x80)
    {
        /* find interface out endpoint information */
        phid->eptin = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress;
        phid->in_maxpacket = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].wMaxPacketSize;
        phid->in_poll = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bInterval;

        phid->chin = usbh_alloc_channel(puhost, phid->eptin);
        /* enable channel */
        usbh_hc_open(puhost, phid->chin, phid->eptin,
                    puhost->dev.address, EPT_INT_TYPE,
                    phid->in_maxpacket,
                    puhost->dev.speed);
    }
    else
    {
        /* get interface out endpoint information */
        phid->eptout = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bEndpointAddress;
        phid->out_maxpacket = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].wMaxPacketSize;
        phid->out_poll = puhost->dev.cfg_desc.interface[hidx].endpoint[eptidx].bInterval;

        phid->chout = usbh_alloc_channel(puhost, phid->eptout);
        /* enable channel */
        usbh_hc_open(puhost, phid->chout, phid->eptout,
                    puhost->dev.address, EPT_INT_TYPE,
                    phid->out_maxpacket,
                    puhost->dev.speed);
    }
}
}

```

```
phid->ctrl_state = USB_HID_STATE_IDLE;
return status;
}
```

3.1.2 HID 类请求

HID 类请求实现一些 HID 设备的标准请求，调用函数 `uhost_request_handler`

- SET_IDLE
- SET_PROTOCOL
- GET_REPORT

代码如下：

```
usb_sts_type uhost_request_handler(void *uhost)
{
    usb_sts_type status = USB_WAIT;
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_hid_type *phid = (usbh_hid_type *)puhost->class_handler->pdata;

    switch(phid->ctrl_state)
    {
        case USB_HID_STATE_IDLE:
            phid->ctrl_state = USB_HID_STATE_GET_DESC;
            break;
        case USB_HID_STATE_GET_DESC:
            if(usbh_hid_get_desc(puhost, 9) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_GET_REPORT;
            }
            break;
        case USB_HID_STATE_GET_REPORT:
            if(usbh_hid_get_report(puhost, phid->hid_desc.wItemLength) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_SET_IDLE;
            }
            break;
        case USB_HID_STATE_SET_IDLE:
            if(usbh_hid_set_idle(puhost, 0, 0) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_SET_PROTOCOL;
            }
            break;
        case USB_HID_STATE_SET_PROTOCOL:
            if(usbh_hid_set_protocol(puhost, 0) == USB_OK)
            {
                phid->ctrl_state = USB_HID_STATE_COMPLETE;
            }
    }
}
```

```
        break;
    case USB_HID_STATE_COMPLETE:
        phid->state = USB_HID_INIT;
        status = USB_OK;
        break;
    default:
        break;
}

return status;
}
```

3.1.3 HID 类数据处理

HID 枚举设置完成之后，USB 主机可以请求设备数据，通过调用 `uhost_process_handler` 来处理。

- 通过 `usbh_interrupt_recv` 函数给设备发送 IN 请求
- 通过 `usbh_get_urb_status(puhost, phid->chin)` 获取 IN 请求的状态
- 通过 `phid->in_poll` 判断多长时间发送一次 IN 请求
- 通过 `phid->protocol` 判断是鼠标还是键盘

代码如下：

```
usb_sts_type uhost_process_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_hid_type *phid = (usbh_hid_type *)puhost->class_handler->pdata;
    urb_sts_type urb_status;
    switch(phid->state)
    {
        case USB_HID_INIT:
            phid->state = USB_HID_GET;
            break;

        case USB_HID_GET:
            usbh_interrupt_recv(puhost, phid->chin, phid->buffer, phid->in_maxpacket);
            phid->state = USB_HID_POLL;
            phid->poll_timer = usbh_get_frame(puhost->usb_reg);
            break;

        case USB_HID_POLL:
            if((usbh_get_frame(puhost->usb_reg) - phid->poll_timer) >= phid->in_poll )
            {
                phid->state = USB_HID_GET;
            }
            else
            {

```

```
    urb_status = usbh_get_urb_status(puhost, phid->chin);
    if(urb_status == URB_DONE)
    {
        puhost->urb_state[phid->chin] = URB_IDLE;
        if(phid->protocol == USB_HID_MOUSE_PROTOCOL_CODE)
        {
            usbh_hid_mouse_decode(phid->buffer);
        }
        else if(phid->protocol == USB_HID_KEYBOARD_PROTOCOL_CODE)
        {
            usbh_hid_keyboard_decode(phid->buffer);
        }
    }
    else if(urb_status == URB_STALL)
    {
        if(usbh_clear_endpoint_feature(puhost, phid->eptin, phid->chin) == USB_OK)
        {
            phid->state = USB_HID_GET;
        }
    }
    }
    break;
default:
    break;
}
return USB_OK;
}
```

3.2 MSC 例程

msc_only_fat32 实现一个简单的 mass storage 主机类，仅支持 FAT32 格式。

MSC 类:

```
usbh_class_handler_type uhost_class_handler =
{
    uhost_init_handler,
    uhost_reset_handler,
    uhost_request_handler,
    uhost_process_handler,
    &usbh_msc
};
```


3.2.1 MSC 类初始化

当 USB 设备标准枚举完成之后，会进行初始化，调用 `uhost_init_handler`，调用此函数将解析接到主机上的设备类型，判断是否是 MSC 类型。

通过设备的配置描述信息进行如下流程解析：

- 查找 MSC 接口
- 查找对应接口的端点号
- 初始化对应通道
- 初始化 bulk-only 和 SCSI 状态机

代码如下：

```
usb_sts_type uhost_init_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usb_sts_type status = USB_OK;
    uint8_t if_x, eptidx = 0;
    usbh_msc_type *pmisc = &usbh_msc;
    puhost->class_handler->pdata = &usbh_msc;

    if_x = usbh_find_interface(puhost, USB_CLASS_CODE_MSC, MSC_SUBCLASS SCSI_TRANS,
    MSC_PROTOCOL_BBB);
    if(if_x == 0xFF)
    {
        USBH_DEBUG("Unsupport Device!");
        return USB_NOT_SUPPORT;
    }
    pmisc->protocol = puhost->dev.cfg_desc.interface[if_x].interface.bInterfaceProtocol;

    for(eptidx = 0; eptidx < puhost->dev.cfg_desc.interface[if_x].interface.bNumEndpoints; eptidx++)
    {
        if(puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].bEndpointAddress & 0x80)
        {
            pmisc->eptin = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].bEndpointAddress;
            pmisc->in_maxpacket = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].wMaxPacketSize;
            pmisc->in_poll = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].bInterval;

            pmisc->chin = usbh_alloc_channel(puhost, pmisc->eptin);
            /* enable channel */
            usbh_hc_open(puhost, pmisc->chin, pmisc->eptin,
                        puhost->dev.address, EPT_BULK_TYPE,
                        pmisc->in_maxpacket,
                        puhost->dev.speed);
        }
        else
        {
            pmisc->eptout = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].bEndpointAddress;
            pmisc->out_maxpacket = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].wMaxPacketSize;
```

```
    pmsc->out_poll = puhost->dev.cfg_desc.interface[if_x].endpoint[eptidx].bInterval;

    pmsc->chout = usbh_alloc_channel(puhost, pmsc->eptout);
    /* enable channel */
    usbh_hc_open(puhost, pmsc->chout, pmsc->eptout,
                puhost->dev.address, EPT_BULK_TYPE,
                pmsc->out_maxpacket,
                puhost->dev.speed);
}
}

msc_bot_scsi_init(pmsc);
usbh_set_toggle(puhost, pmsc->chout, 0);
usbh_set_toggle(puhost, pmsc->chin, 0);
return status;
}
```

3.2.2 MSC 类请求

MSC 类请求实现一些 MSC 设备的标准请求，调用函数 `uhost_request_handler`

- GET_MAX_LUN

代码如下：

```
usb_sts_type uhost_request_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_msc_type *pmsc = (usbh_msc_type *)puhost->class_handler->pdata;
    usb_sts_type status = USB_WAIT;
    uint8_t i_index = 0;

    switch(pmsc->ctrl_state)
    {
        case USBH_MSC_STATE_IDLE:
            pmsc->ctrl_state = USBH_MSC_STATE_GET_LUN;
            break;
        case USBH_MSC_STATE_GET_LUN:
            if((status = usbh_msc_get_max_lun(uhost, (uint8_t *)&pmsc->max_lun)) == USB_OK)
            {
                pmsc->max_lun = (pmsc->max_lun & 0xFF) > USBH_SUPPORT_MAX_LUN ?
                USBH_SUPPORT_MAX_LUN:((pmsc->max_lun & 0xFF) + 1);
                USBH_DEBUG("Support max lun %d", pmsc->max_lun);
                for(i_index = 0; i_index < pmsc->max_lun ; i_index++)
                {
                    pmsc->l_unit_n[i_index].pre_state = USB_FAIL;
                    pmsc->l_unit_n[i_index].change = 0;
                    pmsc->l_unit_n[i_index].state = USBH_MSC_INIT;
                }
            }
    }
```

```

    }
}
break;
case USBH_MSC_STATE_ERROR:
    if((usbh_msc_clear_feature(uhost, 0)) == USB_OK)
    {
        pmsc->ctrl_state = USBH_MSC_STATE_GET_LUN;
    }
    break;
default:
    break;
}

return status;
}

```

3.2.3 MSC 类数据处理

HID 枚举设置完成之后，USB 主机可以请求设备数据，通过调用 `uhost_process_handler` 来处理。实现如下命令：

- INQUIRY
- TEST_UNIT_READY
- READ_CAPACITY10
- REQUEST_SENSE

代码如下：

```

usb_sts_type uhost_process_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_msc_type *pmsc = (usbh_msc_type *)puhost->class_handler->pdata;
    uint64_t msize = 0;
    usb_sts_type status;
    switch(pmsc->state)
    {
        case USBH_MSC_INIT:
            if(pmsc->cur_lun < pmsc->max_lun)
            {
                pmsc->l_unit_n[pmsc->cur_lun].ready = MSC_NOT_READY;

                switch(pmsc->l_unit_n[pmsc->cur_lun].state)
                {
                    case USBH_MSC_INIT:
                        pmsc->l_unit_n[pmsc->cur_lun].state = USBH_MSC_INQUIRY;
                        break;
                    case USBH_MSC_INQUIRY:
                        status = usbh_msc_bot_scsi_get_inquiry(uhost, &pmsc->bot_trans, pmsc->cur_lun,

```

```

&pmisc->l_unit_n[pmisc->cur_lun].inquiry);
    if(status == USB_OK)
    {
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_TEST_UNIT_READY;
    }
    else if(status == USB_FAIL)
    {
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_REQUEST_SENSE;
    }
    break;
case USBH_MSC_TEST_UNIT_READY:
    status = usbh_msc_bot_scsi_test_unit_ready(uhost, &pmisc->bot_trans, pmisc->cur_lun);
    if(status == USB_OK)
    {
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_READ_CAPACITY10;
        pmisc->l_unit_n[pmisc->cur_lun].ready = MSC_OK;
    }
    else if(status == USB_FAIL)
    {
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_REQUEST_SENSE;
        pmisc->l_unit_n[pmisc->cur_lun].ready = MSC_NOT_READY;
    }
    break;

case USBH_MSC_READ_CAPACITY10:
    status = usbh_msc_bot_scsi_capacity(uhost, &pmisc->bot_trans, pmisc->cur_lun,
&pmisc->l_unit_n[pmisc->cur_lun].capacity);
    if(status == USB_OK)
    {
        msize = (uint64_t)pmisc->l_unit_n[pmisc->cur_lun].capacity.blk_nbr *
(uint64_t)pmisc->l_unit_n[pmisc->cur_lun].capacity.blk_size;
        USBH_DEBUG("Device capacity: %llu Byte", msize);
        USBH_DEBUG("Block num: %d ", pmisc->l_unit_n[pmisc->cur_lun].capacity.blk_nbr);
        USBH_DEBUG("Block size: %d Byte", pmisc->l_unit_n[pmisc->cur_lun].capacity.blk_size);
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_IDLE;
        pmisc->state = USBH_MSC_IDLE;
        pmisc->cur_lun ++;
    }
    else if(status == USB_FAIL)
    {
        pmisc->l_unit_n[pmisc->cur_lun].state = USBH_MSC_REQUEST_SENSE;
    }
    break;
case USBH_MSC_REQUEST_SENSE:
    status = usbh_msc_bot_scsi_request_sense(uhost, &pmisc->bot_trans, pmisc->cur_lun);
    if(status == USB_OK)

```

```
        {
            pmsc->l_unit_n[pmsc->cur_lun].state = USBH_MSC_IDLE;
            pmsc->cur_lun++;
        }
        else if(status == USB_FAIL)
        {
            pmsc->cur_lun++;
        }
        break;

    case USBH_MSC_BUSY:
        break;
    case USBH_MSC_ERROR:
        break;
    default:
        break;
    }
}
break;
case USBH_MSC_IDLE:
    if(puhost->user_handler->user_application != NULL)
    {
        puhost->user_handler->user_application();
    }
    default:
        break;
}
return USB_OK;
}
```

3.2.4 MSC 读写数据

MSC 读写数据在通过 FATFS 接口进行实现，在 `usbh_msc_diskio.c` 中实现读写函数。

读函数：

```
DRESULT disk_read (
    BYTE pdrv,      /* Physical drive number to identify the drive */
    BYTE *buff,     /* Data buffer to store read data */
    LBA_t sector,   /* Start sector in LBA */
    UINT count      /* Number of sectors to read */
)
{
    usb_sts_type status;

    status = usbh_msc_read(&otg_core_struct.host, sector, count, buff, pdrv);

    if(status == USB_OK)
```

```
    return RES_OK;

    return RES_ERROR;
}
```

写函数:

```
DRESULT disk_write (
    BYTE pdrv,          /* Physical drive number to identify the drive */
    const BYTE *buff, /* Data to be written */
    LBA_t sector,       /* Start sector in LBA */
    UINT count,         /* Number of sectors to write */
)
{
    usb_sts_type status;

    status = usbh_msc_write(&otg_core_struct.host, sector, count, (uint8_t *)buff, pdrv);

    if(status == USB_OK)
        return RES_OK;

    return RES_ERROR;
}
```

3.3 CDC 例程

cdc 实现一个简单的虚拟串口类。

CDC 类:

```
usbh_class_handler_type uhost_cdc_class_handler =
{
    uhost_init_handler,
    uhost_reset_handler,
    uhost_request_handler,
    uhost_process_handler,
    &usbh_cdc
};
```

3.3.1 CDC 类例程初始化

当 USB 设备标准枚举完成之后，会进行初始化，调用 `uhost_init_handler`，调用此函数将解析接到主机上的设备类型。

通过设备的配置描述信息进行如下流程解析：

- 查找 CDC 接口
- 判断接口是否为虚拟串口
- 查找对应接口的端点号
- 初始化对应通道

代码如下：

```
static usb_sts_type uhost_init_handler(void *uhost)
```

```

{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usb_sts_type status = USB_OK;
    uint8_t if_x;
    usbh_cdc_type *pcdc = &usbh_cdc;
    puhost->class_handler->pdata = &usbh_cdc;

    memset((void *)pcdc, 0, sizeof(usbh_cdc_type));
    if_x = usbh_find_interface(puhost, USB_CLASS_CODE_CDC, ABSTRACT_CONTROL_MODEL,
    COMMON_AT_COMMAND);
    if(if_x == 0xFF)
    {
        USBH_DEBUG("cannot find the interface for communication interface class!");
        return USB_NOT_SUPPORT;
    }
    else
    {
        if(if_x < puhost->dev.cfg_desc.cfg.bNumInterfaces)
        {
            USBH_DEBUG("switching to interface (##d)", if_x);
            USBH_DEBUG("class      : %xh", puhost->dev.cfg_desc.interface[if_x].interface.bInterfaceClass);
            USBH_DEBUG("subclass : %xh", puhost->dev.cfg_desc.interface[if_x].interface.bInterfaceSubClass );
            USBH_DEBUG("protocol : %xh", puhost->dev.cfg_desc.interface[if_x].interface.bInterfaceProtocol );
            if(puhost->dev.cfg_desc.interface[if_x].interface.bInterfaceClass ==
            COMMUNICATION_INTERFACE_CLASS_CODE)
            {
                USBH_DEBUG("CDC device!");
            }
        }
        else
        {
            USBH_DEBUG("cannot select this interface.");
        }

        /* collect the notification endpoint address and length */
        if(puhost->dev.cfg_desc.interface[if_x].endpoint[0].bEndpointAddress & 0x80)
        {
            pcdc->common_interface.notif_endpoint =
            puhost->dev.cfg_desc.interface[if_x].endpoint[0].bEndpointAddress;
            pcdc->common_interface.notif_endpoint_size =
            puhost->dev.cfg_desc.interface[if_x].endpoint[0].wMaxPacketSize;
        }

        /* allocate the length for host channel number in */
        pcdc->common_interface.notif_channel = usbh_alloc_channel(puhost,
        pcdc->common_interface.notif_endpoint);
    }
}

```

```

/* enable channel */
usbh_hc_open(puhost,
             pcdc->common_interface.notif_channel,
             pcdc->common_interface.notif_endpoint,
             puhost->dev.address,
             EPT_INT_TYPE,
             pcdc->common_interface.notif_endpoint_size,
             puhost->dev.speed);

usbh_set_toggle(puhost, pcdc->common_interface.notif_channel, 0);

if_x = usbh_find_interface(puhost, DATA_INTERFACE_CLASS_CODE, RESERVED,
                           NO_CLASS_SPECIFIC_PROTOCOL_CODE);
if(if_x == 0xFF)
{
    USBH_DEBUG("cannot find the interface for data interface class!");
    return USB_NOT_SUPPORT;
}
else
{
    /* collect the class specific endpoint address and length */
    if(puhost->dev.cfg_desc.interface[if_x].endpoint[0].bEndpointAddress & 0x80)
    {
        pcdc->data_interface.in_endpoint = puhost->dev.cfg_desc.interface[if_x].endpoint[0].bEndpointAddress;
        pcdc->data_interface.in_endpoint_size =
            puhost->dev.cfg_desc.interface[if_x].endpoint[0].wMaxPacketSize;
    }
    else
    {
        pcdc->data_interface.out_endpoint =
            puhost->dev.cfg_desc.interface[if_x].endpoint[0].bEndpointAddress;
        pcdc->data_interface.out_endpoint_size =
            puhost->dev.cfg_desc.interface[if_x].endpoint[0].wMaxPacketSize;
    }

    if(puhost->dev.cfg_desc.interface[if_x].endpoint[1].bEndpointAddress & 0x80)
    {
        pcdc->data_interface.in_endpoint = puhost->dev.cfg_desc.interface[if_x].endpoint[1].bEndpointAddress;
        pcdc->data_interface.in_endpoint_size =
            puhost->dev.cfg_desc.interface[if_x].endpoint[1].wMaxPacketSize;
    }
    else
    {
        pcdc->data_interface.out_endpoint =
            puhost->dev.cfg_desc.interface[if_x].endpoint[1].bEndpointAddress;
    }
}

```



```

        pcdc->data_interface.out_endpoint_size =
        puhost->dev.cfg_desc.interface[if_x].endpoint[1].wMaxPacketSize;
    }

    /* allocate the length for host channel number in */
    pcdc->data_interface.in_channel = usbh_alloc_channel(puhost, pcdc->data_interface.in_endpoint);
    /* allocate the length for host channel number out */
    pcdc->data_interface.out_channel = usbh_alloc_channel(puhost, pcdc->data_interface.out_endpoint);

    /* enable in channel */
    usbh_hc_open(puhost,
        pcdc->data_interface.in_channel,
        pcdc->data_interface.in_endpoint,
        puhost->dev.address,
        EPT_BULK_TYPE,
        pcdc->data_interface.in_endpoint_size,
        puhost->dev.speed);

    /* enable out channel */
    usbh_hc_open(puhost,
        pcdc->data_interface.out_channel,
        pcdc->data_interface.out_endpoint,
        puhost->dev.address,
        EPT_BULK_TYPE,
        pcdc->data_interface.out_endpoint_size,
        puhost->dev.speed);

    usbh_set_toggle(puhost, pcdc->data_interface.in_channel, 0);
    usbh_set_toggle(puhost, pcdc->data_interface.out_channel, 0);

    pcdc->state = CDC_IDLE_STATE;

    }
}
return status;
}

```

3.3.2 CDC 类请求

CDC 类请求实现一些 CDC 设备的标准请求，调用函数 `uhost_request_handler`

- GET_LINECODING

代码如下：

```

static usb_sts_type uhost_request_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_cdc_type *pcdc = (usbh_cdc_type *)puhost->class_handler->pdata;
    usb_sts_type status = USB_WAIT;

```

```
status = get_linecoding(uhost, &pcdc->linecoding);

return status;
}
```

3.3.3 CDC 类数据处理

CDC 枚举设置完成之后，USB 主机可以请求设备数据，通过调用 `uhost_process_handler` 来处理。

- 通过 `set_linecoding` 和 `get_linecoding` 配置 `linecoding`
- 通过 `cdc_process_reception(puhost)` 函数给设备发送 IN 请求
- 通过 `cdc_process_transmission(puhost)` 函数发送数据给设备

代码如下：

```
static usb_sts_type uhost_process_handler(void *uhost)
{
    usbh_core_type *puhost = (usbh_core_type *)uhost;
    usbh_cdc_type *pcdc = (usbh_cdc_type *)puhost->class_handler->pdata;
    usb_sts_type status;

    switch(pcdc->state)
    {
        case CDC_IDLE_STATE:
            status = USB_OK;
            break;

        case CDC_SET_LINE_CODING_STATE:
            status = set_linecoding(puhost, pcdc->puserlinecoding);
            if(status == USB_OK)
                pcdc->state = CDC_GET_LAST_LINE_CODING_STATE;
            break;

        case CDC_GET_LAST_LINE_CODING_STATE:
            status = get_linecoding(puhost, &(pcdc->linecoding));
            if(status == USB_OK)
            {
                pcdc->state = CDC_IDLE_STATE;
                if(((pcdc->linecoding.line_coding_b.char_format ==
                    pcdc->puserlinecoding->line_coding_b.char_format)&&
                    (pcdc->linecoding.line_coding_b.data_bits == pcdc->puserlinecoding->line_coding_b.data_bits)&&
                    (pcdc->linecoding.line_coding_b.parity_type ==
                    pcdc->puserlinecoding->line_coding_b.parity_type)&&
                    (pcdc->linecoding.line_coding_b.data_baudrate ==
                    pcdc->puserlinecoding->line_coding_b.data_baudrate)))
                {

```

```
        /* line coding changed */
    }
}

pcdc->state = CDC_IDLE_STATE;
break;

case CDC_TRANSFER_DATA:
    cdc_process_transmission(puhost);
    cdc_process_reception(puhost);
    break;

case CDC_ERROR_STATE:
    status = usbh_clear_ept_feature(puhost, 0, pcdc->common_interface.notif_channel);
    if(status == USB_OK)
        pcdc->state = CDC_IDLE_STATE;
    break;

default:
    break;
}

return status;
}
```

3.4 two_otg_host_demo 例程

AT32F435/437 存在两个 OTGFS，此例程说明同时使用两个 OTGFS 实现两个 host 功能。

- 实现功能：
 - OTGFS1 实现 MSC 主机（参考 MSC 例程）
 - OTGFS2 实现 HID 主机（参考 HID 例程）

3.4.1 配置与实现

在 usb_conf.h 中配置 OTGFS1 和 OTGFS2:

```
#define USE_OTG_HOST_MODE

/**
 * @brief select otgfs1 or otgfs2 define
 */
#define USB_ID 0
#define OTG_CLOCK CRM_OTGFS1_PERIPH_CLOCK
#define OTG_IRQ OTGFS1_IRQn
#define OTG_IRQ_HANDLER OTGFS1_IRQHandler
#define OTG_WKUP_IRQ OTGFS1_WKUP_IRQn
#define OTG_WKUP_HANDLER OTGFS1_WKUP_IRQHandler
#define OTG_WKUP_EXINT_LINE EXINT_LINE_18
```

```

#define OTG_PIN_GPIO                GPIOA
#define OTG_PIN_GPIO_CLOCK          CRM_GPIOA_PERIPH_CLOCK

#define OTG_PIN_DP                  GPIO_PINS_12
#define OTG_PIN_DP_SOURCE           GPIO_PINS_SOURCE12

#define OTG_PIN_DM                  GPIO_PINS_11
#define OTG_PIN_DM_SOURCE           GPIO_PINS_SOURCE11

#define OTG_PIN_VBUS                GPIO_PINS_9
#define OTG_PIN_VBUS_SOURCE         GPIO_PINS_SOURCE9

#define OTG_PIN_ID                  GPIO_PINS_10
#define OTG_PIN_ID_SOURCE           GPIO_PINS_SOURCE10

#define OTG_PIN_SOF_GPIO            GPIOA
#define OTG_PIN_SOF_GPIO_CLOCK      CRM_GPIOA_PERIPH_CLOCK
#define OTG_PIN_SOF                 GPIO_PINS_8
#define OTG_PIN_SOF_SOURCE          GPIO_PINS_SOURCE8

#define OTG_PIN_MUX                 GPIO_MUX_10

#define OTG_PIN_POWER_SWITCH_GPIO   GPIOH
#define OTG_PIN_POWER_SWITCH_CLOCK  CRM_GPIOH_PERIPH_CLOCK
#define OTG_PIN_POWER_SWITCH        GPIO_PINS_3

#define USB2_ID                      1
#define OTG2_CLOCK                   CRM_OTGFS2_PERIPH_CLOCK
#define OTG2_IRQ                     OTGFS2_IRQn
#define OTG2_IRQ_HANDLER              OTGFS2_IRQHandler
#define OTG2_WKUP_IRQ                OTGFS2_WKUP_IRQn
#define OTG2_WKUP_HANDLER             OTGFS2_WKUP_IRQHandler
#define OTG2_WKUP_EXINT_LINE         EXINT_LINE_20

#define OTG2_PIN_GPIO                GPIOB
#define OTG2_PIN_GPIO_CLOCK          CRM_GPIOB_PERIPH_CLOCK

#define OTG2_PIN_DP                  GPIO_PINS_15
#define OTG2_PIN_DP_SOURCE           GPIO_PINS_SOURCE15

#define OTG2_PIN_DM                  GPIO_PINS_14
#define OTG2_PIN_DM_SOURCE           GPIO_PINS_SOURCE14

#define OTG2_PIN_VBUS                GPIO_PINS_13

```

```
#define OTG2_PIN_VBUS_SOURCE          GPIO_PINS_SOURCE13

#define OTG2_PIN_ID                    GPIO_PINS_12
#define OTG2_PIN_ID_SOURCE            GPIO_PINS_SOURCE10

#define OTG2_PIN_SOF_GPIO              GPIOA
#define OTG2_PIN_SOF_GPIO_CLOCK        CRM_GPIOA_PERIPH_CLOCK
#define OTG2_PIN_SOF                    GPIO_PINS_4
#define OTG2_PIN_SOF_SOURCE            GPIO_PINS_SOURCE4

#define OTG2_PIN_MUX                    GPIO_MUX_12

#define OTG2_PIN_POWER_SWITCH_GPIO     GPIOB
#define OTG2_PIN_POWER_SWITCH_CLOCK    CRM_GPIOB_PERIPH_CLOCK
#define OTG2_PIN_POWER_SWITCH           GPIO_PINS_10

#ifndef USE_OTG_HOST_MODE
#ifndef USB_HOST_CHANNEL_NUM
#define USB_HOST_CHANNEL_NUM            16
#endif
#endif

/**
 * @brief usb host mode fifo
 */
/* otg1 host fifo */
#define USBH_RX_FIFO_SIZE                128
#define USBH_NP_TX_FIFO_SIZE             96
#define USBH_P_TX_FIFO_SIZE              96

/* otg2 host fifo */
#define USBH2_RX_FIFO_SIZE                128
#define USBH2_NP_TX_FIFO_SIZE             96
#define USBH2_P_TX_FIFO_SIZE              96

/* usb host vbus power switch */
#define USBH_5V_POWER_SWITCH
#endif
```

在 main 中初始化:

```
int main(void)
{
    nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);

    system_clock_config();
}
```

```

at32_board_init();

/* usb gpio config */
usb_gpio_config();

#ifdef USB_LOW_POWER_WAKUP
    usb_low_power_wakeup_config();
#endif

uart_print_init(115200);

/* enable otgfs clock */
crm_periph_clock_enable(OTG_CLOCK, TRUE);
crm_periph_clock_enable(OTG2_CLOCK, TRUE);

/* select usb 48m clcok source */
usb_clock48m_select(USB_CLK_HEXT);

/* enable otgfs irq */
nvic_irq_enable(OTG_IRQ, 0, 0);
nvic_irq_enable(OTG2_IRQ, 0, 0);

/* init otg1 host */
usbh_init(&otg_core_struct,
          USB_FULL_SPEED_CORE_ID,
          USB_ID,
          &uhost_msc_class_handler,
          &usbh_user_handle);

/* init otg2 host */
usbh_init(&otg2_core_struct,
          USB_FULL_SPEED_CORE_ID,
          USB2_ID,
          &uhost_hid_class_handler,
          &usbh_user_handle);

while(1)
{
    usbh_loop_handler(&otg_core_struct.host);
    usbh_loop_handler(&otg2_core_struct.host);
}
}

```

4 版本历史

表 11 文档版本历史

日期	版本	变更
2021.11.05	2.0.0	最初版本
2022.08.15	2.0.1	1. 增加双OTGFS例程 2. 修改一些错误描述 3. 增加CDC例程
2024.03.26	2.0.2	1. 修改描述错误

重要通知 - 请仔细阅读

买方自行负责对本文所述雅特力产品和服务的选择和使用，雅特力概不承担与选择或使用本文所述雅特力产品和服务相关的任何责任。

无论之前是否有过任何形式的表示，本文档不以任何方式对任何知识产权进行任何明示或默示的授权或许可。如果本文档任何部分涉及任何第三方产品或服务，不应被视为雅特力授权使用此类第三方产品或服务，或许可其中的任何知识产权，或者被视为涉及以任何方式使用任何此类第三方产品或服务或其中任何知识产权的保证。

除非在雅特力的销售条款中另有说明，否则，雅特力对雅特力产品的使用和/或销售不做任何明示或默示的保证，包括但不限于有关适销性、适合特定用途（及其依据任何司法管辖区的法律的对应情况），或侵犯任何专利、版权或其他知识产权的默示保证。

雅特力产品并非设计或专门用于下列用途的产品：（A）对安全性有特别要求的应用，例如：生命支持、主动植入设备或对产品功能安全有要求的系统；（B）航空应用；（C）航天应用或航天环境；（D）武器，且/或（E）其他可能导致人身伤害、死亡及财产损失的应用。如果采购商擅自将其用于前述应用，即使采购商向雅特力发出了书面通知，风险及法律责任仍将由采购商单独承担，且采购商应独立负责在前述应用中满足所有法律和法规要求。

经销的雅特力产品如有不同于本文档中提出的声明和/或技术特点的规定，将立即导致雅特力针对本文所述雅特力产品或服务授予的任何保证失效，并且不应以任何形式造成或扩大雅特力的任何责任。

© 2024 雅特力科技 保留所有权利